

ROD CUTTING (Dynamic Programming)

1. PROBLEM STATEMENT

Given a rod of length n and an array $p[1..n]$ where $p[i]$ is the price of a rod piece of length i , the task is to cut the rod into pieces (or not cut at all) such that the total selling price is maximum.

2. THEORY

- This problem has **Optimal Substructure**.
- It can be solved using **Dynamic Programming**.
- Let $r[n]$ be the maximum profit obtainable from a rod of length n .
- Recurrence Relation:

$$r[n] = \max_{1 \leq i \leq n} \{ p[i] + r[n-i] \}$$

- Base Case: $r[0] = 0$
- We build the solution for $0..n$ bottom-up and store choices to find the optimal cuts.

3. EXAMPLE

Let the length of rod $n = 8$

Price array (1-indexed):

Length (i)	1	2	3	4	6	7	8
Price $p[i]$	1	5	8	9	10	17	20

We have to find maximum profit and optimal cuts.

4. DP TABLE (Bottom-Up Computation)

Rod Length (j)	0	1	2	3	4	6	7	8
Price $p[j]$	0	1	5	8	9	10	17	20
Possible first cut (i) ($1 \leq i \leq j$)	Total Profit for first cut of length i							
1	-	1	1	1	1	1	1	1
2	-	-	5	5	5	5	5	5
3	-	-	-	8	8	8	8	8
4	-	-	-	-	9	9	9	9
5	-	-	-	-	-	10	10	10
6	-	-	-	-	-	-	17	17
7	-	-	-	-	-	-	17	17
8	-	-	-	-	-	-	-	20
$r[j]$ = Maximum Profit	0	1	5	8	10	13	17	22
$s[j]$ = First Cut Length	0	1	2	3	2	2	6	2

How to read the table?

Cell (i, j) shows profit if we first cut a piece of length i and sell it (price $p[i]$) and then solve the remaining length $(j-i)$.

For example:

For $j = 8$ and $i = 2$

$\rightarrow p[2] + r[8-2] = 5 + 17 = 22$
(Best value in column 8)

The maximum in last row $r[j]$ is the answer for length j .

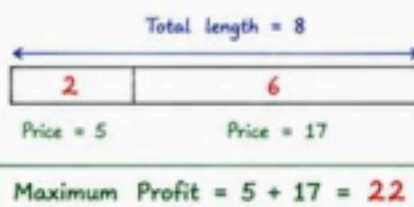
5. FINDING OPTIMAL CUTS

Start from $n = 8$ and repeatedly take $s[n]$

$n = 8 \rightarrow$ first cut = $s[8] = 2 \rightarrow$ remaining length = $8 - 2 = 6$

$n = 6 \rightarrow$ first cut = $s[6] = 6 \rightarrow$ remaining length = $6 - 6 = 0$ (stop)

Optimal cuts: 2, 6



6. FINAL ANSWER

- Maximum Profit = 22
- Optimal Cuts = 2, 6

Observations

- There can be multiple optimal solutions for some inputs.
- DP avoids redundant calculations done in naive recursion.
- Time Complexity: $O(n^3)$
- Space Complexity: $O(n)$

MATRIX-CHAIN MULTIPLICATION (Dynamic Programming)

1. PROBLEM STATEMENT

Given a chain of matrices $A_1, A_2, \dots, A_n$, where matrix A_i has dimension $p_{i-1} \times p_i$, we want to find the most efficient way (i.e., the order) to multiply these matrices together so that the total number of scalar multiplications is minimum.

Example of a chain of 4 matrices:

Let $p = [10, 30, 5, 60, 10]$

Then,

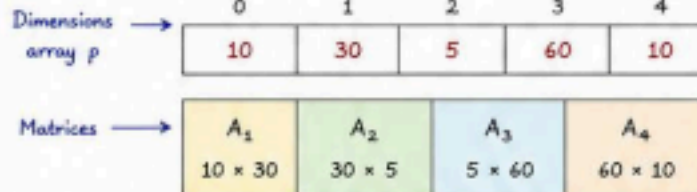
$A_1: 10 \times 30$ $A_2: 30 \times 5$ $A_3: 5 \times 60$ $A_4: 60 \times 10$

We must determine the best way to compute $A_1 A_2 A_3 A_4$.

KEY POINTS

- Overlapping Subproblems
- Optimal Substructure
- DP Approach: Bottom-Up
- Time Complexity: $O(n^3)$
- Space Complexity: $O(n^2)$

CHAIN OF MATRICES



Cost of multiplying two matrices

If A is $p \times q$ and B is $q \times r$, then the cost of AB is $p \times q \times r$ scalar multiplications.

3. EXAMPLE

Let the dimension array be $p = [10, 30, 5, 60, 10]$.

We have 4 matrices A_1, A_2, A_3, A_4 .

We will build the DP table $m[i, j]$ (minimum cost) using the recurrence relation.

Indexing: 1-based for matrices

ORDER OF COMPUTATION (BY CHAIN LENGTH)

Chain Length (L)	Subproblems Computed
1	$m[1,1], m[2,2], m[3,3], m[4,4]$
2	$m[1,2], m[2,3], m[3,4]$
3	$m[1,3], m[2,4]$
4	$m[1,4]$ (final answer)

HOW TO OBTAIN OPTIMAL PARENTHEZIZATION?

Along with table $m[i, j]$, we maintain another table $s[i, j]$ which stores the value of k at which minimum cost is obtained. Then we can construct the optimal parenthesization recursively. Recurrence for $s[i, j]$:

$$s[i, j] = k \text{ that gives minimum in } m[i, j]$$

STEP-BY-STEP COMPUTATION (Summary)

- For $L = 1$: $m[i, i] = 0$ for all i .
- For $L = 2$:
 $m[1, 2] = 10 \times 30 \times 5 = 1500$ $m[2, 3] = 30 \times 5 \times 60 = 9000$ $m[3, 4] = 5 \times 60 \times 10 = 3000$
- For $L = 3$:
 $m[1, 3] = \min \{ m[1,1] + m[2,3] + 10 \times 30 \times 60 = 0 + 9000 + 18000 = 27000,$
 $m[1,2] + m[3,3] + 10 \times 5 \times 60 = 1500 + 0 + 3000 = 4500 \} = 4500 \text{ (k=1)}$
 $m[2, 4] = \min \{ m[2,2] + m[3,4] + 30 \times 5 \times 10 = 0 + 3000 + 1500 = 4500,$
 $m[2,3] + m[4,4] + 30 \times 60 \times 10 = 9000 + 0 + 18000 = 27000 \} = 4500 \text{ (k=2)}$
- For $L = 4$:
 $m[1, 4] = \min \{ m[1,1] + m[2,4] + 10 \times 30 \times 10 = 0 + 4500 + 3000 = 7500,$
 $m[1,2] + m[3,4] + 10 \times 5 \times 10 = 1500 + 3000 + 500 = 5000,$
 $m[1,3] + m[4,4] + 10 \times 60 \times 10 = 4500 + 0 + 6000 = 10500,$
 $m[1,3] + m[4,4] + 10 \times 60 \times 10 = 4500 + 0 + 6000,$
 $\} = 5000 \text{ (k=3)}$

OBSERVATIONS

- Different parenthesizations can give different costs.
- DP helps to avoid redundant computations and finds the optimal order in $O(n^3)$ time.
- Space required is $O(n^2)$ for the tables m and s .

2. THEORY

- Matrix-chain multiplication problem has **Optimal Substructure**.
- It can be solved using **Dynamic Programming**.
- Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute the product $A_i A_{i+1} \dots A_j$.
- If we split the product between A_k and A_{k+1} ($i \leq k < j$), the cost is:

$$m[i, j] = \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} \cdot p_k \cdot p_j \}$$

Base Case: $m[i, i] = 0$ (Only one matrix - no multiplication)

4. DP TABLES (Bottom-Up Computation)

Table 1: $m[i, j]$ - Minimum number of scalar multiplications

$i \setminus j$	1	2	3	4
1	0	1500	4500	5000
2	—	0	9000	27000
3	—	—	0	3000
4	—	—	—	0

Table 2: $s[i, j]$ - Value of k giving minimum

$i \setminus j$	1	2	3	4
1	—	1	1	3
2	—	—	2	3
3	—	—	—	3
4	—	—	—	—

5. FINAL ANSWER

- Minimum number of scalar multiplications = $m[1, 4] = 5000$
- Optimal parenthesization:
 Use table $s[i, j]$ to construct.
 $s[1, 4] = 3 \rightarrow$ split between A_3 and A_4
 So, $(A_1 A_2 A_3) A_4$
 Now for $A_1 A_2 A_3$, $s[1, 3] = 1 \rightarrow$ split between A_1 and $A_2 A_3$
 So, $A_1 (A_2 A_3)$
 Now for $A_2 A_3$, $s[2, 3] = 2 \rightarrow (A_2 A_3)$
Optimal Parenthesization: $A_1 (A_2 A_3) A_4$

COST VERIFICATION

Compute according to $(A_1 (A_2 A_3)) A_4$

1. $A_2 A_3$: cost = $30 \times 5 \times 60 = 9000 \rightarrow$ result size 30×60
2. $A_1 (A_2 A_3)$: cost = $10 \times 30 \times 60 = 18000 \rightarrow$ result size 10×60
3. $(A_1 (A_2 A_3)) A_4$: cost = $10 \times 60 \times 10 = 6000$

Total cost = $9000 + 18000 + 6000 = 5000$

ELEMENTS OF DYNAMIC PROGRAMMING

Dynamic Programming (DP) is an algorithmic technique for solving problems by breaking them into overlapping subproblems and solving each subproblem only once.

The essential elements (ingredients) of Dynamic Programming are the following:

1. OPTIMAL SUBSTRUCTURE

- The problem can be solved optimally by combining optimal solutions of its subproblems.
- An optimal solution contains optimal solutions to its subproblems.

Example:

In the 0/1 Knapsack problem, the optimal set of items for capacity W either includes the last item or not. In both cases, the remaining part is again an optimal solution of a smaller knapsack problem.

2. OVERLAPPING SUBPROBLEMS

- The problem can be broken down into subproblems that are solved multiple times if we use a simple recursive solution.
- DP solves each subproblem once and stores the result for future use.

Example:

In Fibonacci sequence, $F(5)$ requires $F(4)$ and $F(3)$. $F(4)$ again requires $F(3)$ and $F(2)$. So, $F(3)$ is solved more than once in naive recursion.

3. STATE DEFINITION

- A state represents a subproblem.
- It is defined by a set of variables (parameters) that capture all the information required to solve the subproblem.

Example:

In Matrix Chain Multiplication, a state is defined by two indices (i, j) representing the subchain $A_i \dots A_j$.

4. RECURRENCE RELATION (TRANSITION)

- A mathematical relation is defined to express the solution of a state in terms of solutions of smaller states.
- This is the heart of DP.

Example:

Fibonacci: $F(n) = F(n-1) + F(n-2)$ for $n \geq 2$
with base cases: $F(0) = 0, F(1) = 1$

5. BASE CASE (INITIALIZATION)

- The simplest subproblems whose solutions are known in advance.
- These are used to build solutions of larger problems.

Example:

Fibonacci: $F(0) = 0, F(1) = 1$
Matrix Chain Multiplication: $m[i, i] = 0$ (cost of one matrix is 0)

6. ORDER OF COMPUTATION

- The order in which states are solved so that when a state is computed, all the states it depends on are already computed.
- Can be done in:
 - Top-Down (Memoization)** - Solve recursively and store.
 - Bottom-Up (Tabulation)** - Solve iteratively from small subproblems to large ones.

Example:

In Bottom-Up Fibonacci, we compute from $F(2), F(3), \dots, F(n)$ using already computed previous values.

7. STORAGE OF SUBPROBLEM SOLUTIONS

- Store the solution of each state after computing it.
- This can be done using arrays, tables, or maps.
- Avoids recomputation and saves time.

Example:

In DP table $dp[i][j]$ (in LCS problem), we store the length of LCS for prefixes $X[1..i]$ and $Y[1..j]$.

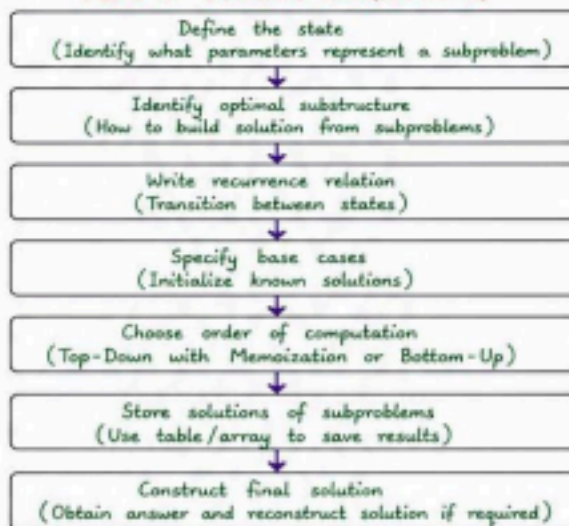
8. CONSTRUCTION OF FINAL SOLUTION

- After filling the table (or computing with memoization), the final answer is obtained from the required state.
- Sometimes, we also reconstruct the actual solution by tracing back the choices.

Example:

In 0/1 Knapsack, after filling the table, the answer is in $dp[n][W]$. We can backtrack to find which items are taken.

FLOW OF DYNAMIC PROGRAMMING



SUMMARY TABLE

	Element	Purpose	Example (Fibonacci)
1	Optimal Substructure	Optimal solution from optimal subproblems	$F(5) = F(4) + F(3)$
2	Overlapping Subproblems	Same subproblems solved again and again	$F(3)$ computed in $F(5), F(4)$ and $F(3)$
3	State Definition	Define what a subproblem is	State = n (compute $F(n)$)
4	Recurrence Relation	Relation between states	$F(n) = F(n-1) + F(n-2)$
5	Base Case	Known solutions to start with	$F(0) = 0, F(1) = 1$
6	Order of Computation	Order to solve all states	$n = 2$ to n
7	Storage of Solutions	Save results of states	$dp[n]$ array
8	Construct Final Solution	Get final answer (and actual solution if needed)	Answer = $F(n)$

BENEFITS OF DP

- Reduces time complexity from exponential to polynomial.
- Avoids recomputation of the same subproblems.
- Systematic approach for optimization problems.
- Widely applicable (Knapsack, LCS, Matrix Chain Multiplication, Edit Distance, etc.)

NOTE:

Not every problem can be solved using DP. The problem must have Optimal Substructure and Overlapping Subproblems.

LONGEST COMMON SUBSEQUENCE (LCS) – DYNAMIC PROGRAMMING

1. PROBLEM STATEMENT

Given two sequences X and Y, find the length of their Longest Common Subsequence (LCS) and print one LCS.

Subsequence: A sequence that can be derived from another sequence by deleting some (or no) characters without changing the order of the remaining characters.

Example:

X = "AGGTAB" Y = "GXTXAYB"

One LCS: "GTAB" Length of LCS = 4

2. THEORY

- This problem has Optimal Substructure.
- It can be solved using Dynamic Programming.
- Let $X = x_1 x_2 \dots x_m$ and $Y = y_1 y_2 \dots y_n$
- Let $L[i][j]$ be the length of LCS of $X[1..i]$ and $Y[1..j]$.
- **Recurrence Relation:**

$$L[i][j] = \begin{cases} L[i-1][j-1] + 1 & \text{if } x_i = y_j \\ \max(L[i-1][j], L[i][j-1]) & \text{if } x_i \neq y_j \end{cases}$$

Base Case:

If $i = 0$ or $j = 0$, then $L[i][j] = 0$

(If any one string is empty, LCS length is 0.)

3. KEY POINTS

- Overlapping Subproblems
- Optimal Substructure
- Use DP (Bottom-Up)
- Time Complexity: $O(m \times n)$
- Space Complexity: $O(m \times n)$
(Can be optimized to $O(\min(m, n))$ for length only)

NOTATION USED

m = length of X
 n = length of Y
 $L[i][j]$ = length of LCS of $X[1..i]$ and $Y[1..j]$

4. EXAMPLE WITH DP TABLE (BOTTOM-UP)

Let

	Y →	0	1	2	3	4	5	6	7
		-	G	X	T	X	A	Y	B
X ↓	0	0	0	0	0	0	0	0	0
1 A	0	0	← 1	← 1	← 1	← 1	← 1	← 1	← 1
2 G	0	↑ 1	← 1	← 1	← 1	← 1	← 1	← 1	← 1
3 G	0	↑ 1	← 1	← 1	← 1	← 1	← 1	← 1	← 1
4 T	0	↑ 1	↑ 1	↖ 2	← 2	← 2	← 2	← 2	← 2
5 A	0	↑ 1	↑ 1	↑ 2	↑ 2	↖ 3	← 3	← 3	← 3
6 B	0	↑ 1	↑ 1	↑ 2	↑ 2	↑ 3	↖ 3	← 4	← 4

We build a table $L[0..m][0..n]$.
 Rows correspond to X (including empty prefix) and columns correspond to Y (including empty prefix).

- ↖ Diagonal (match) : $x_i = y_j \rightarrow L[i][j] = L[i-1][j-1] + 1$
 ↑ Up (from top) : $L[i][j] = L[i-1][j]$
 ← Left (from left) : $L[i][j] = L[i][j-1]$

Interpretation:

$L[6][7] = 4$
 Length of LCS = 4

5. BACKTRACKING TO FIND ONE LCS

Start from $L[m][n] = L[6][7]$ and move backwards according to the arrows.

Step	(i, j)	X[i-1]	Y[j-1]	Direction Taken	LCS so far
1	(6, 7)	B	B	↖ Diagonal (match)	B
2	(5, 6)	A	Y	← Left	B
3	(5, 5)	A	A	↖ Diagonal (match)	AB
4	(4, 4)	T	X	↑ Up	AB
6	(3, 3)	G	X	↑ Up	AB
7	(2, 3)	G	T	← Left	AB
8	(2, 2)	G	X	← Left	AB
9	(2, 1)	G	G	↖ Diagonal (match)	GAB

Reverse the collected characters → "B A T G"

One LCS (forward) : "G T A B"

6. COMPLEXITY

- Time Complexity: $O(m \times n)$
(We fill a table of size $(m+1) \times (n+1)$, each cell in $O(1)$ time.)
- Space Complexity: $O(m \times n)$
(Can be optimized to $O(\min(m, n))$ space if only length is needed.)

For X = "AGGTAB" and Y = "GXTXAYB"

Length of LCS = 4

One LCS = "GTAB"

- NOTES:**
- There can be multiple LCSs of same length. Another LCS for this example is "GTAB".
 - DP ensures we get the optimal length and can backtrack to get one LCS.

OPTIMAL BINARY SEARCH TREES (OBST) – DYNAMIC PROGRAMMING

1. PROBLEM STATEMENT

Given a sorted sequence of keys $K = \{k_1 < k_2 < \dots < k_n\}$ with successful search frequencies $f_1, f_2, \dots, f_n$ and unsuccessful search frequencies $d_0, d_1, \dots, d_n$ (where d_i is the frequency of searching a key between k_i and k_{i+1}), construct a Binary Search Tree with minimum expected search cost.

What is Expected Search Cost?

Cost of a search is the number of comparisons.
Expected Search Cost =
(Sum of (frequency $\times$ depth of that event))

- k_i : actual keys (sorted)
- f_i : frequency of successful search for k_i
- d_i : frequency of unsuccessful search between k_i and k_{i+1} ($i = 0$ to n)
- n : number of keys

2. THEORY

An optimal BST has the property of Optimal Substructure. If we consider keys from k_i to k_j as a subtree, the part of the tree formed by these keys must also be an optimal BST for these keys.

Let $e[i][j]$ = minimum expected cost of BST containing keys $k_i, k_{i+1}, \dots, k_j$

$w[i][j]$ = total frequency (weight) for keys k_i to k_j (including d 's)

Recurrence Relations:

$$w[i][j] = w[i][j-1] + f_j + d_j \quad \text{for } i \leq j$$

$$e[i][i-1] = d_{i-1} \quad \text{for all } i$$

$$e[i][j] = \min_{r=i \dots j} \{e[i][r-1] + e[r+1][j] + w[i][j]\}$$

Root of $e[i][j]$ is the r which gives minimum cost.

Time Complexity: $O(n^3)$

3. NOTATION

- $e[i][j]$: Minimum cost for keys from k_i to k_j
- $w[i][j]$: Total weight for keys from k_i to k_j
- $root[i][j]$: Root key index which gives minimum cost for keys from k_i to k_j

4. EXAMPLE

Consider 5 keys.

Keys (sorted): 10, 20, 30, 40, 50
(k_1 k_2 k_3 k_4 k_5)

Successful search frequencies (f):

3 2 1 1 2
(f_1 f_2 f_3 f_4 f_5)

Unsuccessful search frequencies (d):

2 3 1 1 1 2
(d_0 d_1 d_2 d_3 d_4 d_5)

5. WEIGHT TABLE $w[i][j]$

$w[i][j]$ = total frequency (f 's and d 's) for keys k_i to k_j

$i \backslash j$	1	2	3	4	5
1	7	12	14	16	20
2	-	6	8	10	14
3	-	-	4	6	10
4	-	-	-	4	8
5	-	-	-	-	4

(Note: $w[i][i] = f_i + d_{i-1} + d_i$)

6. DP TABLE $e[i][j]$ (Minimum Expected Cost)

$e[i][j]$ = minimum cost for keys k_i to k_j

$i \backslash j$	1	2	3	4	5
1	7	19	21	24	30
2	-	6	14	17	24
3	-	-	4	10	18
4	-	-	-	4	12
5	-	-	-	-	4

How to fill?

- Length = $j - i + 1$
- First fill for length = 1 (diagonal)
- Then length = 2, then 3, and so on.
- For each $e[i][j]$, try all roots $r = i$ to j and take minimum.

7. ROOT TABLE $root[i][j]$

$root[i][j]$ = root index which gives minimum cost

$i \backslash j$	1	2	3	4	5
1	1	1	2	2	2
2	-	2	2	2	2
3	-	-	3	3	5
4	-	-	-	4	5
5	-	-	-	-	5

($root[i][j] = r$ means key k_r is root for keys $k_i \dots k_j$)

8. HOW $e[i][j]$ IS COMPUTED? (INFO FOR ANY CELL)

Example: To compute $e[1][3]$

Keys: k_1, k_2, k_3 Weight $w[1][3] = 14$

Choose root (r)	Cost = $e[i][r-1] + e[r+1][j] + w[i][j]$	Total Cost
$r = 1$	$e[1][0] + e[2][3] + 14$ $= 2 + 8 + 14$	24
$r = 2$	$e[1][1] + e[3][3] + 14$ $= 7 + 4 + 14$	25
$r = 3$	$e[1][2] + e[4][3] + 14$ $= 19 + 1 + 14$	21

Minimum cost is **21** with root = 3 ($k_3 = 30$)
Therefore $e[1][3] = 21$ and $root[1][3] = 3$

Base Cases:

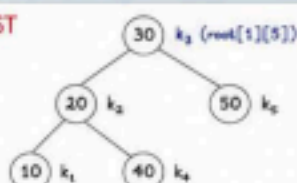
- $e[i][i-1] = d_{i-1}$
(cost when no key, only unsuccessful search)
- $e[i][i] = d_{i-1} + f_i + d_i$

Order of Computation (by length)

Length 1: $e[i][i]$
Length 2: $e[i][i+1]$
Length 3: $e[i][i+2]$
...
Length n : $e[1][n]$

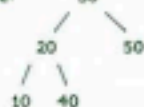
9. CONSTRUCTING THE OPTIMAL BST

- The overall root is $root[1][n]$.
- For left subtree: use $root[1][r-1]$.
- For right subtree: use $root[r+1][n]$.
- Recur for each subtree.



Optimal BST for the given example

Structure:



Minimum Expected Search Cost = 44

Expected Search Cost

$$\begin{aligned}
 &= 3 \times 3 + 2 \times 2 + 1 \times 1 + 1 \times 2 + 2 \times 2 \\
 &\quad + 2 \times 3 + 3 \times 2 + 1 \times 2 + 1 \times 2 + 1 \times 2 + 2 \times 3 \\
 &= 9 + 4 + 1 + 2 + 4 \\
 &\quad + 6 + 6 + 2 + 2 + 2 + 6 \\
 &= 44
 \end{aligned}$$

10. SUMMARY

- OBST minimizes the expected search cost considering both successful and unsuccessful searches.
- Uses DP with two tables: $w[i][j]$ and $e[i][j]$ (and $root[i][j]$).
- Recurrence:

$$e[i][j] = \min_{r=i \dots j} \{e[i][r-1] + e[r+1][j] + w[i][j]\}$$

- Time Complexity: $O(n^3)$
- Space Complexity: $O(n^2)$

12. KEY TAKEAWAYS

- OBST is an excellent application of Dynamic Programming.
- Considers both successful (f) and unsuccessful (d) searches.
- Produces a BST with minimum expected comparisons.
- Useful in database indexing, compilers, and search optimization.

11. ALGORITHM (BOTTOM-UP DP)

- Initialize $w[i][i-1] = d_{i-1}$ and $e[i][i-1] = d_{i-1}$ for all i .
- For length = 1 to n
For $i = 1$ to $n - \text{length} + 1$
 $j = i + \text{length} - 1$
 $w[i][j] = w[i][j-1] + f_j + d_j$
 $e[i][j] = \min_{r=i \dots j} \{e[i][r-1] + e[r+1][j] + w[i][j]\}$
 $root[i][j] = r$ which gives minimum
- The answer is $e[1][n]$. Construct tree using root table.

